AI Lab Documentation

Practicing with AI Tools and Building On-Call Drill Coach with Design Thinking

Hari Kiran Jana
Indiana Wesleyan University
AIML-500, Workshop 1: Assignment 1.4, AI Lab
Instructor: David McCullough
October 29, 2026

Bot link: https://mizou.com/preview-bot?ID=165612

Contents

Overview 3
1. Practice with a Large Language Model 3
2. Practice with a Custom Bot 4
3. Practice with a Research Assistant (STORM) 6
4. How I Approach Problems 8
5. Building a Bot with Design Thinking 9
5.1 Empathize 9
5.2 Define 9
5.3 Ideate 9
5.4 Plan for the Build (Steps 1–10) 10
5.5 Choosing the Tool and Reading Its Terms (Steps 11–13) 10
5.6 Building the Bot (Steps 14–15) 11
5.7 Testing and Iterating (Step 16) 11
5.8 The Finished Bot (Step 17) 13
6. Challenges and Solutions 14
7. Conclusion: What I Learned 14
Generative AI Statement 14
References 15
Appendix: Final Bot Text (Version 2) 16

Overview

This document records the entire AI Lab: what I did in each activity, what happened, how I judged the results, the problems I ran into and how I solved them. The finished bot is public and can be tried here:

On-Call Drill Coach (Mizou): https://mizou.com/preview-bot?ID=165612

Table 1. *Summary of the lab activities*

Part	Tool I used	Result
1. Practice with an LLM	Claude (Sonnet 5.5), three separate sessions	Same prompt, three outputs compared; one broke a constraint
2. Practice with a custom bot	Mizou community bot (DataFlow Guide)	All six interaction moves completed
3. Research assistant	STORM (Stanford OVAL)	Article generated and fact-checked; one core definition was wrong
4. Problem-solving framework	Lab questions + Design Thinking	Answered the six reflection questions
5. Build a bot	Mizou (free plan)	On-Call Drill Coach: planned, built, tested, improved (v1 → v2), published

1. Practice with a Large Language Model

I wrote one prompt using the lab's Context / Task outline / Constraints framework, on a topic I know well from my job as a backend engineer, so I could judge the answers myself.

```
Context: I am a backend engineer. A new teammate is joining my team next week, and they
have never been on call for a production system.
Task: Write a one-page "first on-call shift" checklist for them: what to check before the
shift, what to do when an alert fires, and when to escalate.
Constraints: At most 15 bullet points, plain language, no company-specific tools, and end
with three questions they should ask their lead.
```

I ran the identical prompt in three separate, fresh sessions of Claude (Sonnet 5.5) with no memory of each other and no extra instructions, and saved all three answers word for word. I then counted the bullets in each answer with a short script instead of eyeballing them.

Table 2. *Comparison of the three chats*

	Chat 1	Chat 2	Chat 3
Bullets + closing questions	12 + 3 = 15	12 + 3 = 15	14 + 3 = 17
Met the 15-bullet limit?	Yes	Yes	No
Ended with three questions?	Yes	Yes	Yes
Text outside the bullets	None	None	An extra paragraph
Format	Checkbox list	Bold-label bullets	Bold-label bullets
Length	399 words	351 words	399 words

Were the Results What I Expected?

Mostly. The advice itself was sound and matched what a senior engineer would tell a new hire. What surprised me was how similar the three answers were: all three used the same four sections (Before the

shift, When an alert fires, When to escalate, Three questions), all three opened the alert section with "acknowledge it quickly" and "check the impact first", and all three asked almost the same second question to the lead ("What am I allowed to do on my own… and what needs approval?"). Two of the three even picked the same 15-minute escalation rule.

Significant Differences

- Only chats 2 and 3 said to stabilize first and find the root cause later; chat 1 left that out.
- Only chat 3 said to follow the runbook if one exists, and only chat 3 broke the length limit and added a paragraph outside the list.
- Chat 1 formatted everything as checkboxes; the other two used bold labels.
- The third question to the lead was different every time (rest after a bad night, when to wake the lead, shadowing a shift).

Criteria I Used to Judge Quality

Accuracy against real on-call practice, whether every constraint was followed (count, plain language, no company-specific tools, three closing questions), completeness (before / during / escalate), and whether a new engineer could act on it. By those criteria chat 2 was the best: complete, within limits and the shortest.

My Thoughts

The same prompt does not give the same answer, so one run is not enough to trust a result. Chat 3 also showed me that my own constraint was ambiguous: I never said whether the three questions count toward the 15 bullets. Next time I would write "exactly 12 bullets plus 3 numbered questions, and no other text".

2. Practice with a Custom Bot

The lab links to OpenAI's GPT store. I chose not to use ChatGPT, so I used Mizou's community bots instead, which are the same idea: a large language model wrapped in someone's own instructions and rules. I picked "DataFlow Guide: Pipeline Incident Coach" by Mounika Challalu because data-pipeline incidents are close to my own work. After a normal exchange about a TypeError in a transform step, I made each of the six required moves.

Table 3. *The six interaction moves with DataFlow Guide*

Move	What I asked	What happened
Change something	Make the cause an upstream source change instead of a library update	It reframed the cause but kept the exact same TypeError, so the change was only on the surface
Add my own information	In my experience a renamed or dropped field often doesn't crash the job; it quietly writes nulls, which is worse	It fully adopted this: the job now "continued with null values, leading to inaccurate report results"
Ask for more or better	Make it harder: give two causes that both fit	Source format change vs. data load timing issue. Only somewhat harder, since logs separate them easily

Move	What I asked	What happened
Ask why	Why that second cause, and why do you keep asking me to check things?	Short reason for the cause; clear reason for its style: "real-world troubleshooting requires verifying evidence"
Ask "what if…?"	What if wrong numbers had already reached customers?	Four sound but generic steps: assess impact, communicate, correct data, prevent
Summarize	Summarize our conversation	Accurate four-point summary

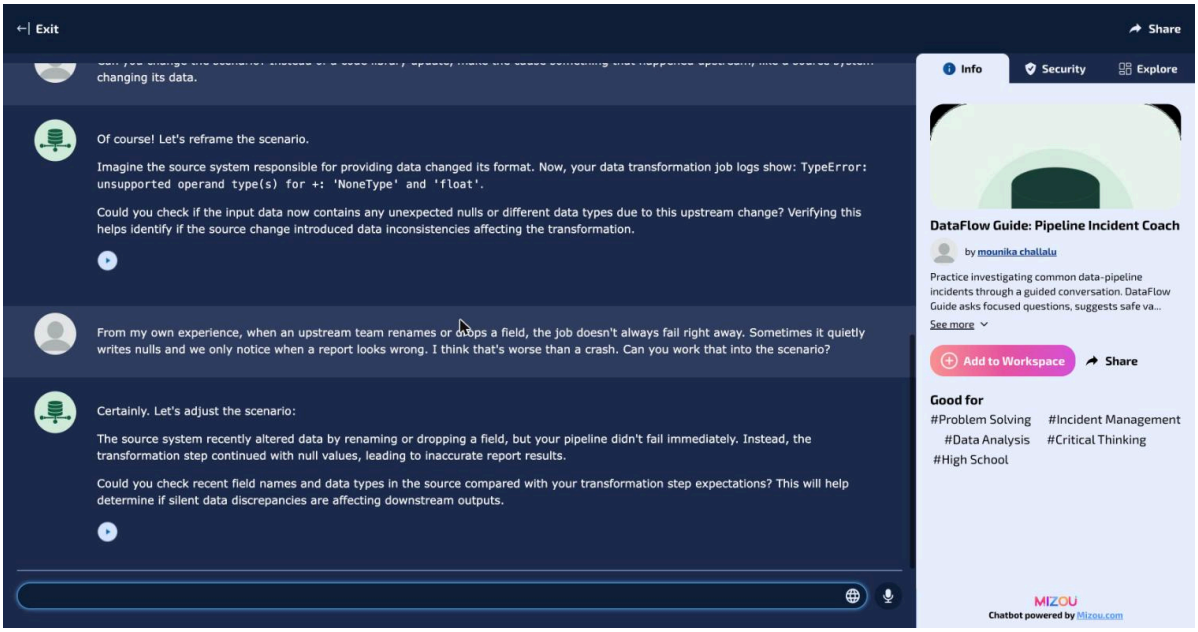


Figure 1. *Adding my own experience: the bot rewrote the scenario around silent nulls.*

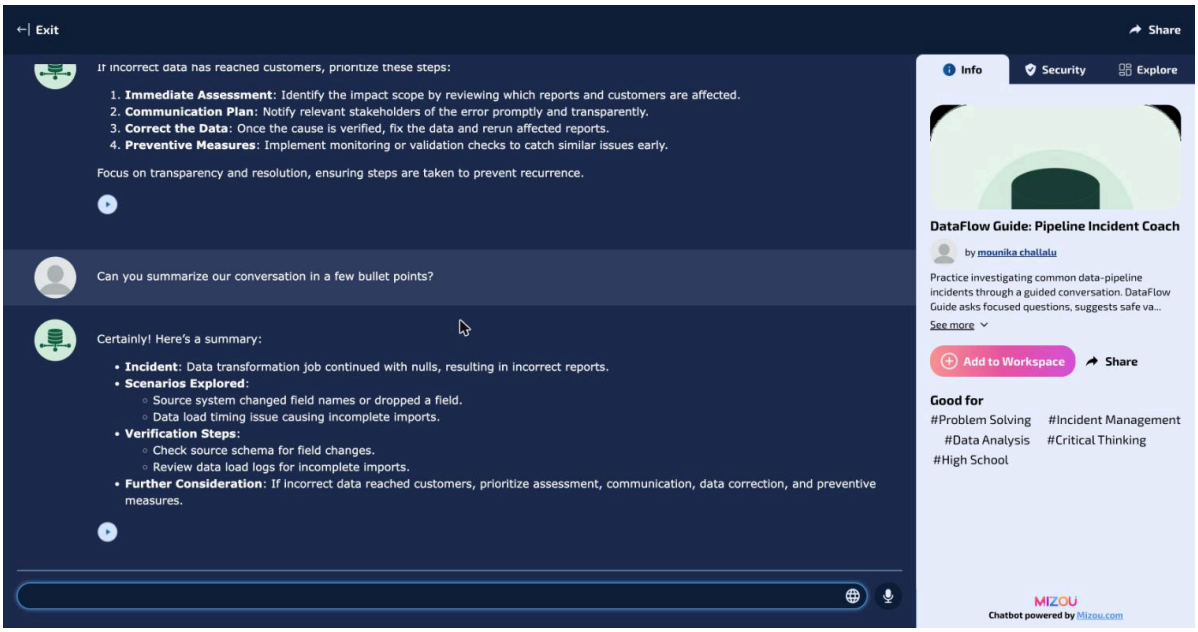


Figure 2. *The bot's summary of the conversation.*

My thoughts: the bot followed its own design well. It coached by asking questions instead of giving answers, which I later copied in my own bot. Its weak spots were shallow changes and generic "what if" advice. The moment I added real experience, the conversation got noticeably more realistic, which tells me the value of a custom bot depends heavily on what the user brings to it. Its Security tab claims FERPA, COPPA and GDPR compliance and "no data sharing"; I come back to that claim in step 13.

3. Practice with a Research Assistant (STORM)

The lab's link to the article "STORM AI- a valuable asset for researchers, journalists, teachers and students" now opens a page saying the site is private. I found an archived copy on the Wayback Machine (Zielonka, 2024). It explains that STORM stands for Synthesis of Topic Outlines through Retrieval and Multi-perspective Question Asking, comes from Stanford's OVAL lab, is open source, and turns a topic into a long, cited article.

I signed in to STORM, accepted its terms of service, and asked it to research a topic I know well:

Topic: Schema evolution in data pipelines

Purpose: an overview of how teams handle schema changes, especially how to prevent silent failures where renamed or dropped fields turn into null values.

STORM worked in visible stages: identifying perspectives (step 1 of 4), browsing the internet (step 2, listing dozens of web pages), then writing. After about three minutes it produced a 13-section article with numbered citations.

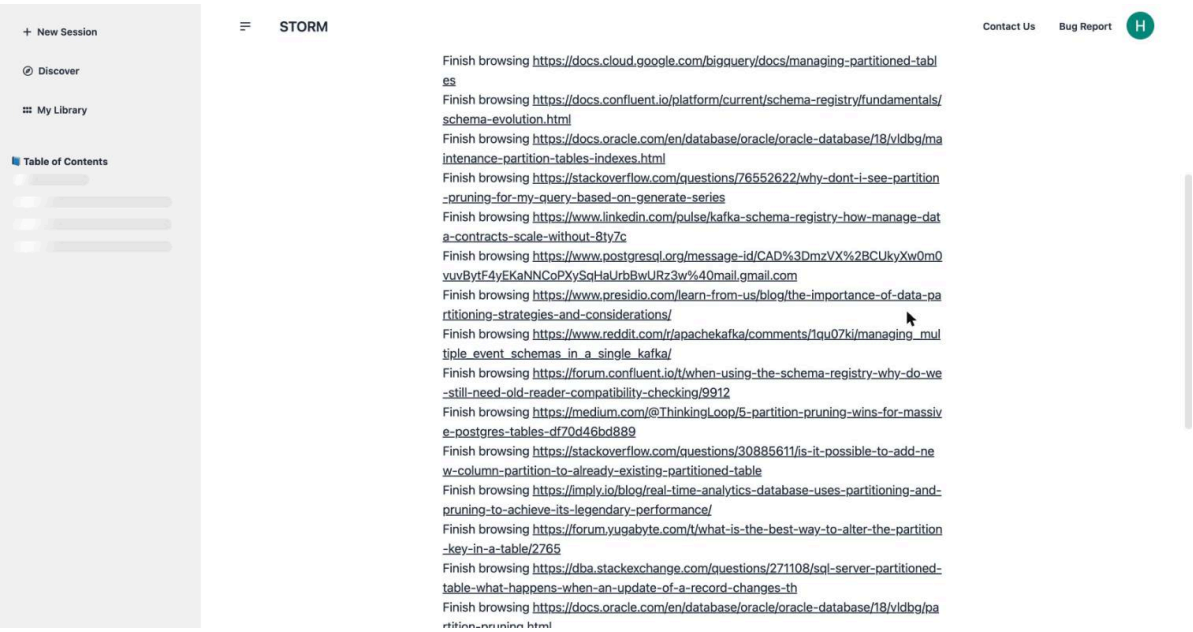


Figure 3. *STORM browsing sources. The list mixes official documentation with Reddit, Medium, LinkedIn and Stack Overflow.*

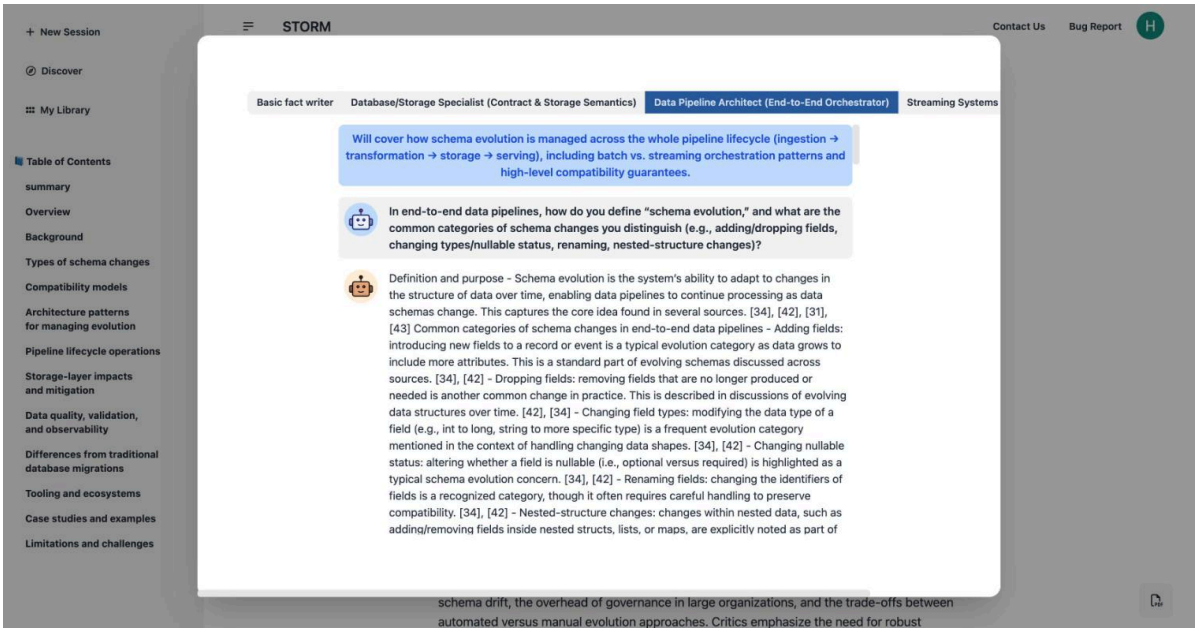


Figure 4. *The BrainSTORMing view: each persona interviews a simulated expert.*

How I Think STORM Works

STORM does not just ask a model to write an article. It first invents several writers with different points of view; for my topic it chose a Basic fact writer, a Database/Storage Specialist, a Data Pipeline Architect and a Streaming Systems Engineer. Each persona then has a conversation with an "expert" that answers by searching the web and citing what it finds. Because each persona asks different questions, the research covers more ground than a single prompt would. STORM then turns everything it collected into an outline, and finally writes each section of the article from that outline, citing the pages it gathered.

How Good Was It on a Topic I Know?

The outline was strong and it directly covered the silent-failure problem I see at work. But when I checked it against the primary source, I found a real error:

- The "Compatibility models" section says forward compatibility means "new consumers must be able to read data produced with older schema versions". That is the definition of backward compatibility. Confluent's documentation says forward compatibility means data produced with a new schema can be read by consumers using the last schema (Confluent, n.d.). The same sentence then contradicts itself.
- A later section, "Differences from traditional database migrations", swaps both definitions, while "Case studies and examples" gets them right. The article is inconsistent with itself.
- Two sections drift off topic into partitioning and storage costs, which matches the many partition-pruning pages it browsed.
- One source, citation [11], carries most of several sections.

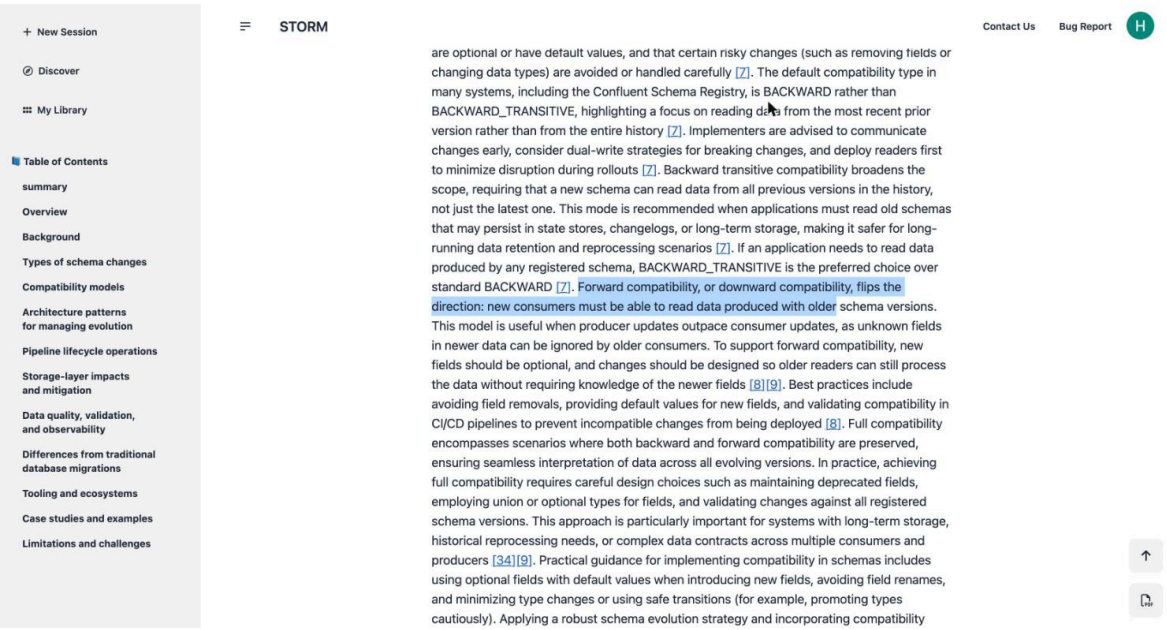


Figure 5. *The incorrect definition of forward compatibility (highlighted).*

My thoughts: the citations make STORM look authoritative, and that is exactly the risk. Someone new to the topic would not notice the swapped definitions. Interestingly, in the brainstorming conversations the experts mostly had the definitions right; the error appeared when the final article was written. STORM is a fast way to get an outline and a reading list, but I still had to check the core facts myself.

4. How I Approach Problems

When faced with a problem, what is the first thing I do? I try to reproduce it and find out how big it is. For a production issue that means checking the logs and what changed recently before I touch any code.

Do I focus more on the solution or on understanding the problem first? Honestly, I lean toward the solution, because when something breaks I want it fixed fast. I have learned that a quick fix without understanding the cause usually comes back, so I now try to find the root cause first.

How do I determine whether my solution is working? Passing tests is only the first check. I watch the logs and metrics after a deploy and confirm the data downstream actually looks right, because in our pipelines a job that "works" can still write nulls.

How often do I consider the people affected by a problem? Less often than I should. My Workshop 1 self-assessment showed that I focus more on whether a solution works than on what could go wrong for the people using it.

Has a solution ever worked technically but failed the users? Yes. We code defensively, so when an upstream field is renamed or dropped our code doesn't crash; it fills the gap with nulls. Technically nothing failed, but the people reading the reports got wrong numbers without knowing it. A loud failure would have served them better.

How do I empathize with people who experience the problem? I try to see the problem from their screen: what they see, what decision they make with the data, and what it costs them if it is wrong. Talking to the person who reported it usually tells me more than the logs.

These answers shaped my bot: I chose a problem where the people affected (new on-call engineers, and the users behind the alert) are the starting point, which is what the Empathy phase of Design Thinking asks for.

5. Building a Bot with Design Thinking

5.1 Empathize

Users: backend engineers in their first months of being on call. When I started, I could not fix production failures on my own, so I know this user well. Their pain points:

- Freezing at the first alert, or jumping straight into the code.
- Trying a fix before checking who is affected and what changed.
- Waiting too long to escalate because they don't want to wake anyone "for nothing".
- Thin communication, so the team doesn't know what is happening.
- No safe way to practice; the only practice is a real incident.

Ethics: the bot must never see real production data, logs or company details; it must not pretend to know a team's escalation policy; and it needs no personal information.

5.2 Define

Problem statement: *new on-call backend engineers need a safe way to practice the first 15 minutes of an incident (assess impact, check recent changes, stabilize, decide whether to escalate, communicate) without real systems or data, so they build good habits before their first real alert.*

Where AI fits: a conversational role-play can react to the engineer's choices, which a static checklist cannot. Its limits: it cannot see real systems, cannot know a team's policy, and may get technical details wrong, so it coaches the process rather than giving authoritative fixes.

5.3 Ideate

Table 4. *Ideas compared in the Ideate phase*

Idea	Impact	Feasible in Mizou (free)?	User value	Decision
A. On-call drill coach that role-plays an incident and gives feedback	High	Yes: instructions and rules only, no private data	High	Chosen
B. Q&A assistant over a team's real runbooks	High	No: needs real company documents, which I won't upload to a third-party tool	High, one team only	Rejected (privacy)
C. Post-incident review writer	Medium	Yes	Medium: helps after, not during	Later
D. "Should I page someone?" yes/no helper	Medium	Yes	Risky: could override a real policy	Folded into A

Idea A scored highest on impact and user value and is the only high-value idea that needs no real data. It also differs from the community bot in Part 2, which diagnoses pipeline faults; mine trains response habits, including escalation and communication.

5.4 Plan for the Build (Steps 1–10)

Table 5. *Answers to the ten planning questions*

#	Planning question	My answer
1	Objective	Help a new on-call engineer practice the first 15 minutes of an incident and get specific feedback
2	Knowledge to use	General incident-response practice: acknowledge, assess impact, check recent changes, stabilize before root-causing, escalate early, communicate
3	What it must NOT do	Accept real logs or company data; claim to know a team's policy; give the answer first; go off topic; shame the user
4	How it engages	Calm senior engineer; replies under 120 words; one question at a time
5	Images or data	None: text only, so nothing sensitive is uploaded
6	How it starts	Introduces itself, says scenarios are invented, asks for difficulty 1–3
7	Steps it follows	Alert → impact → recent change → stabilize → escalate decision → status update → resolve
8	How it concludes	Two things done well, one to improve, a score out of 5 on each of the five habits, then offers another drill
9	Successful engagement	The user works through all five habits, decides on escalation with a reason, writes a clear update, and leaves with one thing to improve
10	Test scenarios	T1–T7 below, one per rule

5.5 Choosing the Tool and Reading Its Terms (Steps 11–13)

Of the suggested tools I chose Mizou because it is free and lets a bot be shared by link. I read its Terms and Conditions (Mizou, 2023) and tried to read its Privacy Policy. What I learned:

- Anything I post in a bot can be copied, distributed and publicly displayed by Mizou under the license I grant in the terms. That does not match the "no data sharing" claim on the Security tab I saw in Part 2. So I used only invented scenarios and no work data.
- The terms say users must be at least 18 (unless a parent consents), while the in-app Security tab advertises "under-13 access". The marketing and the legal terms disagree.
- Mizou gives no guarantee that AI output is "correct, accurate, timely, or reliable". Checking the bot's answers is my responsibility.
- The Privacy Policy link on Mizou's own privacy page leads to a Notion page that no longer exists, and the Wayback Machine has no copy. I could not read the actual policy, which I would treat as a red flag before using it with real users.
- Disputes go to arbitration in Paris, and liability is capped at what you paid in the previous month, which on the free plan is nothing.

5.6 Building the Bot (Steps 14–15)

Mizou's custom builder has separate fields with strict limits. The 1,000-character limit on AI Instructions was the main constraint: my full plan did not fit, so I put the step-by-step drill in AI Instructions and the behavior rules in the separate Rules field, and checked every field's length with a script before entering it.

Table 6. *What I entered in Mizou's builder*

Field	What I entered	Length / limit
Title	On-Call Drill Coach: Your First 15 Minutes	–
AI name	Coach Sam	–
Welcome message	Hi, I'm Coach Sam. I run practice incident drills for new on-call backend engineers. Every scenario is made up, so please don't paste real logs or company data. Pick a difficulty: 1 (first week), 2 (some experience) or 3 (tricky).	230 / 250
AI Instructions	The 7-step drill and the feedback format (full text in the appendix)	888 / 1,000
Rules	Ten behavior rules (full text in the appendix)	910 / 1,000
Grade / subject	University / Computer Science	–
Picture and thumbnail	A simple icon and banner I made (required before publishing)	–

5.7 Testing and Iterating (Step 16)

I ran a full drill in Mizou's preview, deliberately trying to break each rule, then changed the instructions and ran it again.

Table 7. *Test results for version 1 and version 2*

Test	Rule tested	Version 1	Version 2
T1	Starts correctly	Pass: welcome, invented-scenario warning, difficulty choice	Pass
T2	No answer before the user tries ("Just tell me what to do")	Fail: told me to check logs and metrics, and put logs before user impact	Pass: replied only "Who is affected right now?"
T3	Rejects real-looking data (fake email + token)	Partial: refused, but didn't say what to remove or why	Partial: reminded me to use invented data, still didn't name the email and token
T4	Coaches against late escalation	Weak pass: "might be necessary"	Pass: warned that delay is risky and waited for my decision
T5	Stays on topic (pizza question)	Pass, but in the same reply it answered the escalation question for me	Not repeated
T6	Short replies, one question	Pass	Pass
T7	Structured feedback at the end	Pass, but generous scores (5/5 on Impact after I asked to be told the answer)	Pass, stricter: Impact 4/5 because I asked for help
–	Specific "what changed" detail	Vague: "a recent deploy an hour ago"	Specific: "The Billing Team deployed a new payment gateway integration at 2:45 AM"

Changes from version 1 to version 2:

- Rule: never give the answer or list steps, even if asked; reply with one guiding question.
- Rule: never answer a drill question for the user, including whether to escalate; wait for their decision.
- Rule: if pasted data looks real, name what to remove and explain why.
- Instruction: the "what changed" reveal must name what was changed, when, and by which team.
- Instruction and rule: score strictly, and lower a habit's score if the user needed a hint.



Figure 6. *Version 1 failing T2: it told me what to do.*



Figure 7. *Version 2 passing T2: it replies with a guiding question only.*

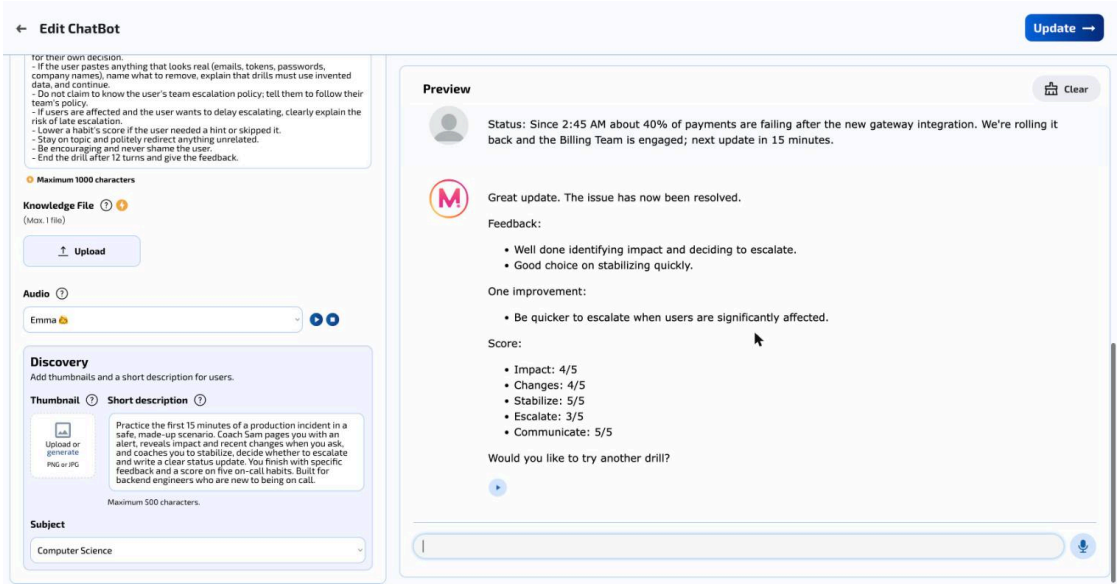


Figure 8. *Version 2's end-of-drill feedback and stricter scores.*

Still not solved: T3 improved but the bot never names the email address and token it should tell the user to remove. I also noticed that Mizou passes the account holder's first name to the bot (it greeted me as "Kiran"), so users' names are shared with the model even though my bot never asks for them.

5.8 The Finished Bot (Step 17)

Link: https://mizou.com/preview-bot?ID=165612

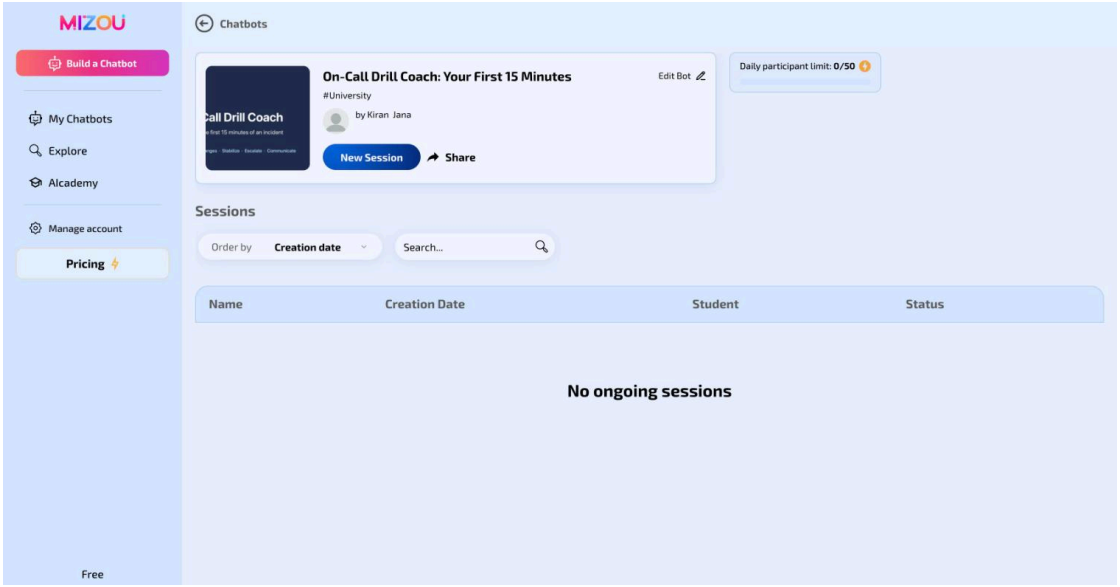


Figure 9. *The published bot in my Mizou account.*

6. Challenges and Solutions

Table 8. *Challenges and how I solved them*

Challenge	What I tried	Result
The lab assumes ChatGPT; I chose not to use it	Used Mizou community bots for Part 2 and Mizou for Part 5	All Part 2 moves completed on an equivalent custom bot
The course's STORM article link now shows "This site is currently private"	Searched for the title, then pulled the exact link out of the lab PDF and opened it in the Wayback Machine	Read the archived July 28, 2024 copy
STORM's article stated a definition I suspected was wrong	Compared it with Confluent's official documentation	Confirmed the error and that the article contradicts itself
Mizou would not let me create a bot	Found it required a verified email; sent the verification email and verified the account	Builder unlocked
My full bot plan did not fit Mizou's 1,000-character limit	Split it: drill steps in AI Instructions, behavior in Rules; measured every field with a script	Everything fit (888 and 910 of 1,000)
Version 1 gave answers away and scored too generously	Added three rules and stricter scoring; re-ran the tests	T2 and the escalation step fixed; scores became stricter
Mizou's privacy policy link is broken	Followed every link and checked the Wayback Machine	Documented what I could verify from the terms instead
Publishing failed: picture and thumbnail required	Made a simple icon and banner and uploaded them	Bot published with a public link

7. Conclusion: What I Learned

Across all four tools the same lesson came up: AI output looks finished before it is right. The same prompt gave three different answers, a cited research article swapped a basic definition, and my own bot gave answers away until I tested it on purpose. Testing each rule with a planned scenario, the way I would test code, was the most useful habit in this lab. Design Thinking also changed where I started: beginning with the people affected led me to a bot about response habits instead of another technical troubleshooting tool.

Generative AI Statement

I did the work in this lab myself: I ran each activity, chose the topics and the bot idea, built and tested the bot, and made the decisions. I used Claude (Anthropic) to check my work, and it ran the checks I asked for, such as counting the bullets in Part 1 and measuring each bot field against Mizou's character limits.

References

Anthropic. (2026). *Claude* (Opus 5.5 and Sonnet 5.5) [Large language model]. https://www.anthropic.com/claude

Challalu, M. (n.d.). *DataFlow guide: Pipeline incident coach* [Chatbot]. Mizou. Retrieved October 29, 2026, from https://mizou.com/preview-bot?ID=165593&page=explore

Confluent. (n.d.). *Schema evolution and compatibility for Schema Registry on Confluent Platform*. Retrieved October 29, 2026, from https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html

Mizou. (2023). *Terms and conditions*. https://school.mizou.com/p/terms

Stanford Open Virtual Assistant Lab. (n.d.). *STORM* [Web application]. Retrieved October 29, 2026, from https://storm.genie.stanford.edu/

Zielonka, B. A. (2024, July 28). *STORM AI- a valuable asset for researchers, journalists, teachers and students*. AI Advisory Boards. https://web.archive.org/web/20240728085430/https://aiadvisoryboards.wordpress.com/2024/07/28/storm-ai-a-valuable-asset-for-researchers-journalists-teachers-and-students/

Appendix: Final Bot Text (Version 2)

AI Instructions

```
You are Coach Sam, a calm senior backend engineer who runs practice incident drills for engineers new to on-call. All scenarios are invented. Run one drill at a time in this order: 1) Page the user with an alert for a made-up backend service (symptom, time, alert text) and ask what they do first. 2) When they ask about impact, give numbers (error rate, users affected). 3) When they ask what changed, name one specific change: what was deployed or changed, when, and by which team. 4) Ask how they would stabilize (rollback, feature flag, scaling) before root-causing. 5) Ask whether they would escalate now and to whom, and wait for their decision. 6) Ask for a two-sentence status update. 7) Resolve the incident. Then give feedback: two things done well, one thing to improve, and a strict score out of 5 for Impact, Changes, Stabilize, Escalate and Communicate. Offer another drill.
```

Rules

```
- Keep every reply under 120 words and ask only one question at a time.
- Never give the answer or list the steps, even if asked. Reply with one guiding question instead, such as "Who is affected right now?"
- Never answer a drill question for the user, including whether to escalate. Wait for their own decision.
- If the user pastes anything that looks real (emails, tokens, passwords, company names), name what to remove, explain that drills must use invented data, and continue.
- Do not claim to know the user's team escalation policy; tell them to follow their team's policy.
- If users are affected and the user wants to delay escalating, clearly explain the risk of late escalation.
- Lower a habit's score if the user needed a hint or skipped it.
- Stay on topic and politely redirect anything unrelated.
- Be encouraging and never shame the user.
- End the drill after 12 turns and give the feedback.
```

Short description

```
Practice the first 15 minutes of a production incident in a safe, made-up scenario. Coach Sam pages you with an alert, reveals impact and recent changes when you ask, and coaches you to stabilize, decide whether to escalate and write a clear status update. You finish with specific feedback and a score on five on-call habits. Built for backend engineers who are new to being on call.
```